

团队分工（6人）

- 抓包模块（2人）：Scapy配置、BPF过滤、性能优化。
- 协议解析（2人）：各层头部解析、校验和验证、会话跟踪。
- 过滤统计与GUI（2人）：实时过滤、统计图表、界面布局。

文献调研重点

- Scapy文档、Ethernet帧格式 (IEEE 802.3)、IP协议 (RFC 791)、TCP协议 (RFC 793)、UDP协议 (RFC 768)、BPF过滤器语法。
- 参考文献不少于10篇。

提交物

完整代码、协议解析文档、抓包示例（PCAP文件可选）、演示视频。

【工具建议】 核心库为 Scapy；GUI框架可选用 PyQt5/6（功能强大）或 Tkinter（内置，简单）；作为学习参考，可对比 Wireshark 的解析逻辑。

题目17：基于混合加密的文件安全传输工具

功能需求

- 客户端-服务器模式，TCP传输文件（支持大文件>1GB）。
- 密钥协商**：RSA-2048（或ECC P-256）协商AES-256会话密钥，每次传输使用不同密钥。
- 加密传输**：
 - AES-256-GCM模式（提供加密+完整性校验，优于CBC+HMAC）。
 - 文件分块（每块1MB），每块独立加密，带16字节GCM Tag。
 - 自定义应用层协议：块序号（4字节）+ 块长度（4字节）+ 加密数据 + Tag。
- 可靠性**：
 - 显示传输进度（百分比、速度、ETA）。
 - 断点续传：记录已确认块序号，重连后从断点续传。
 - 服务端支持多客户端并发（线程池或异步）。

网络技术要求

- 使用Python `cryptography` 库实现RSA/AES，理解**混合加密体系**（非对称加密密钥+对称加密数据）。
- 自定义TCP协议**：解决粘包问题（长度字段），实现块确认机制（ACK）。
- 测量加密开销**：对比明文传输 vs AES-256-GCM的吞吐率（MB/s）。
- 可选：使用**TLS 1.3**替代自定义密钥协商，对比实现复杂度。

团队分工（6人）

- TCP通信与协议（2人）：连接管理、分块传输、断点续传、并发控制。

- 加密模块（2人）：RSA密钥生成、AES-GCM实现、密钥协商流程。
- 界面与进度（1人）：CLI或GUI进度条、文件选择。
- 测试与文档（1人）：大文件测试、性能测试、安全分析。

文献调研重点

- RSA算法原理、AES-GCM模式 (NIST SP 800-38D)、混合加密、TCP可靠传输、断点续传协议设计。
- 参考文献不少于10篇。

提交物

客户端代码、服务端代码、协议文档、性能测试报告、大文件传输演示视频。

【工具建议】加密库使用 cryptography；进度条显示可使用 tqdm；断点续传状态可存储在JSON 文件中。

题目18：基于eBPF的网络流量监控与异常告警系统（进阶选做）

功能需求

1. 使用Python `bcc` 库编写eBPF程序，在内核层捕获TCP/UDP数据包（无需拷贝到用户态）。
2. 统计：每个连接的**TCP重传率**、**RTT分布**、**丢包率**（基于TCP SACK信息，可选）。
3. 用户态Python程序：
 - 实时读取eBPF映射（BPF Maps）中的统计信息。
 - 通过WebSocket推送到前端。
4. 前端：动态折线图展示实时吞吐率、柱状图展示Top 10延迟连接，**阈值告警**（重传率>5%时红色警示）。

网络技术要求

- 理解**eBPF架构**：内核态程序（C语言）→ 编译为BPF字节码 → 挂载到 kprobe/tracepoint。
- 使用**BPF Maps**（Hash Table）在内核态与用户态共享数据。
- 对比测试：eBPF vs `scapy` 抓包的**CPU占用率**和**丢包率**（高负载下）。
- 限制：仅需支持Linux内核5.0+，无需处理XDP（数据面）。（注：此题难度较高，适合基础好的团队选做）

团队分工（6人）

- eBPF内核程序（2人）：C语言编写、BPF Maps设计、编译加载。
- 用户态与WebSocket（2人）：数据读取、WebSocket服务、聚合逻辑。
- 前端与测试（2人）：实时图表、告警UI、性能对比测试。

文献调研重点